

Real-Time Customer 360 - Complete Guide

Real-Time Customer 360

Complete Project Guide & Interview Handbook

Prepared for Sudama Pandit

SudamaPandit/real-time-customer-360-data-platform

POSITIONING

The repository demonstrates streaming graph construction, contracts, offline quality checks, aggregation, a dbt model and S3 Terraform. It does not include a running Kafka/Spark stack, quarantine sink, serving database implementation or reconciliation orchestration.

Clickable Contents

1. Executive summary

2. Beginner foundations

3. Architecture diagram

4. End-to-end data flow

5. Repository structure

6. Folder-wise code walkthrough

7. Quality, reliability and security

Clickable Contents (continued)

8. Testing and CI evidence
9. Gaps and critical risks
10. Production-ready redesign
11. Fifty interview questions
12. Rapid revision sheet
13. Glossary

Executive Summary

What this project is

A focused streaming example reads customer events from Kafka with Spark Structured Streaming, parses a defined event schema and writes valid events as event-date-partitioned Parquet. Python modules validate event contracts and offline batches, pandas builds customer summaries, dbt defines a customer-level aggregation, and Terraform provisions a versioned S3 bucket.

Business value

Organizations receive customer activity from web, mobile and business systems. A Customer 360 view summarizes behavior such as logins, purchases, event count and most recent activity so analytics and engagement teams can work with one customer-level dataset.

60-second interview introduction

I built a focused Customer 360 streaming reference. Spark Structured Streaming reads Kafka events, parses a typed schema, filters invalid identifiers and timestamps, and writes valid events to checkpointed, date-partitioned Parquet. I added Python contracts, offline batch quality checks, a pandas aggregation, a dbt customer model and basic S3 Terraform. CI proves contract, quality and aggregation behavior, but the repository does not yet implement quarantine, a serving database or an end-to-end deployed Kafka/Spark environment.

Honest status

The repository demonstrates streaming graph construction, contracts, offline quality checks, aggregation, a dbt model and S3 Terraform. It does not include a running Kafka/Spark stack, quarantine sink, serving database implementation or reconciliation orchestration.

Beginner Foundations

Use this section to learn the vocabulary before studying the code. A data pipeline moves data through controlled stages. ETL means extract from a source, transform into a useful and reliable shape, and load into a destination.

- > Streaming processes unbounded events continuously; micro-batches are small bounded slices inside Spark.
- > Kafka topics are split into partitions; ordering is guaranteed only within one partition.
- > A Spark checkpoint stores offsets and query state so a restarted job can continue.
- > Event time is when the business event occurred; ingest time is when the platform observed it.
- > A Customer 360 is a consolidated customer-level view, not necessarily one physical database table.

How to study the repository

First memorize the business flow, then trace one record through every component. Next learn which parts are actual runtime code, which are offline helpers, and which capabilities are missing. In interviews, evidence matters more than broad claims.

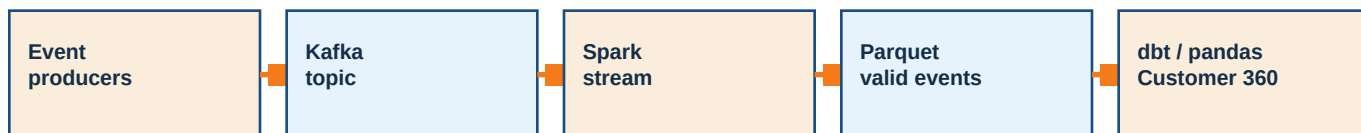
Architecture Diagram

The solid flow is implemented in code. Dashed or lighter notes mark external dependencies or incomplete production capabilities.

How to explain this diagram

- > Producers are expected to send JSON events with IDs, type, timestamp and payload.
- > Kafka provides ordered partitions and decouples producers from the Spark consumer.
- > Spark parses a Struct type and adds ingestion date fields.
- > Only valid rows are written to partitioned Parquet with a checkpoint.
- > dbt and pandas offer separate customer aggregation paths; no serving database load is included.

CHECKPOINT + CONTRACT LAYER: parsing, valid filtering, offline quality tests



End-to-End Data Flow

Step 1 - Event contract

Each event should contain `event_id`, `customer_id`, `event_type`, `event_ts` and `payload`. The Python contract normalizes IDs/type and validates ISO timestamps.

Step 2 - Kafka ingestion

Spark `readStream` subscribes to the topic with `startingOffsets=latest`. That setting processes new data for a fresh checkpoint rather than historical backlog.

Step 3 - JSON parsing

Spark `from_json` applies a `StructType` and selects event fields. `ingest_ts` and `event_date` are derived.

Step 4 - Valid-event filtering

The streaming graph keeps non-null `event_id`, `customer_id` and `event_ts`. Invalid rows are currently dropped, not quarantined.

Step 5 - Stateful continuity and output

`writeStream` writes append-mode Parquet partitioned by `event_date` and stores checkpoint state at the configured checkpoint location.

Step 6 - Customer aggregation

The pandas helper deduplicates `event_id` and calculates `event_count`, `last_event_ts`, `purchase_count`, `login_count` and `customer_status`. The dbt model computes similar SQL metrics from an external silver source.

Repository Structure

This table explains what every important folder contributes and whether it is used by the main runtime path.

Path	Purpose	Runtime truth
.github/workflows/ci.yml	Compile and pytest	Offline CI
src/contracts.py	Single-event validation/normalization	Used by Python callers/tests; not Spark UDF
src/stream_processor.py	Kafka-to-Parquet Spark graph	Main streaming builder; not executed in CI
src/data_quality.py	Offline batch gate	Unit-tested
src/customer_360.py	Pandas customer aggregation	Unit-tested
src/ai_insights.py	Optional aggregate Bedrock summary	Disabled by default
dbt/dbt_project.yml	dbt project configuration	Executable with external profile
dbt/models/customer_360.sql	Customer-level SQL model	Requires silver source
dbt/schema.yml	Source/model tests	Not run in current CI
infra/terraform/main.tf	Versioned S3 bucket	Not applied by CI
requirements-runtime.txt	Spark/Kafka/AWS packages	Not installed by unit CI
tests/	Contracts, quality, aggregation, AI	Runs in CI

Folder-wise Code Walkthrough

src/contracts.py

Validates one Python event, parses ISO time, strips IDs and lowercases the type.

```
required = {"event_id", "customer_id", "event_type", "event_ts", "payload"}
event_ts = datetime.fromisoformat(value.replace("Z", "+00:00"))
event_type = str(event["event_type"]).strip().lower()
```

Interview meaning: This contract is not automatically executed inside the Spark graph; parity tests are needed.

src/stream_processor.py

Builds but does not start a Structured Streaming writer, keeping construction separate from deployment.

```
spark.readStream.format("kafka")
.option("subscribe", topic)
.load()
...
valid.writeStream.format("parquet")
.option("checkpointLocation", checkpoint_path)
```

Interview meaning: The function returns a writer; an external entry point must call start and manage termination.

src/data_quality.py

Calculates counts for blank IDs/types, duplicates and invalid timestamps, and fails on an empty or bad batch.

```
failing = {name: value for name, value in metrics.items()
if name != "row_count" and value}
if not metrics["row_count"] or failing:
raise ValueError(...)
```

Interview meaning: Useful for replays and batch checks, but not wired to the live stream.

src/customer_360.py

Deduplicates event IDs, groups by customer and derives simple engagement metrics.

```
groupby("customer_id").agg(
event_count=("event_id", "count"),
last_event_ts=("event_ts", "max"),
purchase_count=("event_type", lambda s: (s == "purchase").sum()))
```

Interview meaning: This is an offline pandas path and will not scale as the serving implementation.

dbt/models/customer_360.sql

Defines a warehouse-side aggregation over silver.customer_events.

Continued

```
select customer_id, count(*) as event_count,  
max(event_ts) as last_event_ts,  
sum(case when event_type = 'purchase' then 1 else 0 end) as purchase_count  
from {{ source('silver', 'customer_events') }}  
group by customer_id
```

Interview meaning: The external silver source and dbt profile must exist; current CI does not run dbt.

infra/terraform/main.tf

Creates a named S3 bucket and enables versioning.

```
resource "aws_s3_bucket" "customer_360" { ... }  
resource "aws_s3_bucket_versioning" "customer_360" {  
  versioning_configuration { status = "Enabled" }  
}
```

Interview meaning: Terraform covers S3 only, not Kafka, Spark, IAM, logging or networking.

Quality, Reliability and Security

Event contract

Python validation is explicit and produces clear errors. Spark uses a similar but smaller null filter, so contract parity is incomplete.

Checkpointing

The writer accepts a checkpoint location, which is required for recovery. Exactly-once behavior still depends on a stable checkpoint and sink semantics.

Deduplication

The pandas aggregate drops duplicate event_id. The live Spark stream does not watermark/dropDuplicates, so duplicates can reach Parquet.

Late data

event_date comes from event_ts, but no withWatermark/window logic exists. Late-event policy is not implemented.

Quarantine

Invalid streaming records are filtered and lost. A production pipeline should persist the raw value, reason and metadata.

Privacy

payload is represented as a string in Spark and an object in Python. Sensitive payload governance must be explicit before wider use.

Security baseline

No real credentials should be stored in Git. Runtime identities should use least-privilege roles, secrets should come from a managed secret backend, logs should avoid raw personal data, and CI permissions should remain read-only unless a workflow genuinely needs write access.

Testing and CI Evidence

A passing CI badge proves only the checks that CI actually executes. It is not proof of deployment or end-to-end service integration.

Area	Evidence	What it does not prove
Contract	Valid normalization and invalid timestamp/payload/IDs	Spark contract parity
Batch quality	Counts and failures for blanks, duplicates, invalid time, empty input	Live streaming gate
Customer aggregate	Deduplication and purchase/status metrics	Warehouse or serving load
AI config	Disabled path and missing model behavior	Live Bedrock response
CI compile	Python syntax under src	PySpark graph construction
dbt/Terraform	Not run	SQL model validity or infrastructure plan

Best interview wording

I tested the pure Python contract, offline batch quality rules, customer-level aggregation and safe AI configuration behavior. CI passes without Kafka, Spark, AWS or a database. I would not claim an end-to-end stream test; the next evidence is a local Kafka/Spark checkpoint test, dbt build against ephemeral PostgreSQL and Terraform validation.

Tests to add next

- > Local Kafka plus Spark test with valid, malformed, duplicate and late events.
- > Checkpoint restart test proving offsets resume without unintended replay.
- > Contract parity test between Python validation and Spark accepted rows.
- > dbt build/test against an ephemeral warehouse schema.
- > terraform fmt/validate plus a staging plan and policy scan.

Gaps and Critical Risks

These are not reasons to hide the project. They are the strongest discussion points because they show that you can distinguish a portfolio implementation from a production system.

Severity	Risk	Consequence / response
Critical	Invalid stream rows are silently filtered	Data is lost without evidence; write a quarantine sink with reason codes.
High	Live stream does not deduplicate	Retries/producers can create duplicate events; add watermark plus dropDuplicates or sink idempotency.
High	No runnable stream entry point	The graph is built but not started/deployed; add configuration, start/await and packaging.
High	No end-to-end environment	Kafka/Spark/S3 behavior is unproven; add integration and staging evidence.
Medium	Python and Spark contracts differ	Accepted records can vary; create one schema and parity tests.
Medium	No late-event policy	Customer metrics can drift; define watermark, allowed lateness and reconciliation.
Medium	dbt source is external	Model cannot run alone; provide profile/example and integration fixture.
Medium	Terraform covers S3 only	Missing IAM, encryption controls, Kafka, compute, logs and alarms.

Production-Ready Redesign

Phase 1: Make ingestion observable

Persist raw Kafka metadata and invalid records, add structured metrics and define a versioned event contract.

Phase 2: Add streaming correctness

Use stable checkpoints, watermark/deduplication, late-data handling and idempotent downstream publication.

Phase 3: Prove locally

Run Kafka/Spark integration fixtures, restart tests and contract parity tests; run dbt against ephemeral storage.

Phase 4: Complete infrastructure

Provision MSK or an alternative, Spark runtime, S3 encryption/lifecycle, IAM, networking, logs and alarms.

Phase 5: Build serving and reconciliation

Load a scalable customer store/warehouse, reconcile streaming results from durable history and document SLAs.

How to answer design follow-ups

Use Context -> Decision -> Trade-off -> Evidence -> Next step. State what the repository demonstrates, cite the exact test or implementation, acknowledge the boundary, and explain the production change. Never invent deployment history.

Fifty Interview Questions

The answers below are written as spoken interview responses. Learn the ideas rather than memorizing every word.

1 Explain the project end to end.

Interview-ready answer: Customer events arrive in Kafka. Spark parses a defined schema, adds ingest and event dates, filters required fields and writes valid events to checkpointed, date-partitioned Parquet. Python and dbt modules then demonstrate customer-level aggregation.

2 Is this truly real-time?

Interview-ready answer: It uses Spark Structured Streaming APIs, so it represents continuous micro-batch processing. The repository does not include a running cluster or measured latency, so I do not claim a production real-time SLA.

3 Why Kafka?

Interview-ready answer: Kafka decouples producers and consumers, retains ordered event logs per partition and lets the consumer scale by partitions. The deployment and topic configuration are external to this repository.

4 What does `startingOffsets=latest` mean?

Interview-ready answer: For a new checkpoint, Spark begins with events arriving after the query starts. Existing checkpoint state takes precedence. For backfills I would use a separate bounded replay strategy.

5 How does checkpointing help?

Interview-ready answer: Spark stores processed offsets and state at the checkpoint location so a restarted query can resume. The location must be durable, unique per query and protected from accidental reuse or deletion.

6 Do you guarantee exactly once?

Interview-ready answer: No end-to-end guarantee is proven. Spark checkpointing gives strong internal recovery semantics, but source behavior, file sink commits and downstream consumers must all be designed and tested.

7 What happens to invalid events?

Interview-ready answer: The current Spark graph filters them out, which is a known gap. I would write invalid raw values and reason codes to a quarantine path or topic.

8 Why is silent filtering dangerous?

Interview-ready answer: It hides producer defects and makes reconciliation impossible. Operators need reject counts, samples, ownership and replay capability.

Interview Questions (continued)

9 Are duplicates handled live?

Interview-ready answer: No. The pandas aggregation deduplicates event_id, but the live Spark graph does not. I would add watermark plus dropDuplicates or an idempotent sink keyed by event_id.

10 How would you handle late events?

Interview-ready answer: I would define allowed lateness, apply an event-time watermark, update windowed/stateful results within that boundary and route very late data to replay/reconciliation.

11 What is the difference between event_ts and ingest_ts?

Interview-ready answer: event_ts is when the activity occurred; ingest_ts is when the pipeline processed it. Their difference is useful for lag and late-arrival monitoring.

12 Why partition Parquet by event_date?

Interview-ready answer: It aligns storage with time-based replay and analytics, reducing scans. I would monitor file sizes and compact small files.

13 How does the Customer 360 aggregation work?

Interview-ready answer: It groups deduplicated events by customer and calculates count, most recent timestamp, purchase count and login count, then derives a simple status.

14 Why have both pandas and dbt aggregation?

Interview-ready answer: Pandas makes logic easy to unit-test offline; dbt shows the warehouse-side model. They are alternative paths and need parity tests rather than being presented as one deployed flow.

15 Where is the serving database?

Interview-ready answer: It is not implemented. The current outputs are Parquet and transformation examples. A production design would load an analytical warehouse or customer-serving store.

16 Does Terraform deploy the platform?

Interview-ready answer: No. It creates and versions an S3 bucket only. Kafka, Spark, IAM, networking, encryption configuration, logs and alarms remain external.

17 Did you test Spark?

Interview-ready answer: Current CI does not install runtime Spark or construct the streaming graph. I tested pure Python contracts and aggregates; local Spark/Kafka integration is the next test layer.

Interview Questions (continued)

18 How would you test restart recovery?

Interview-ready answer: Start a query with a durable checkpoint, publish known events, stop it, publish more events, restart with the same checkpoint and verify all intended events appear once according to the sink contract.

19 How would you reconcile streaming aggregates?

Interview-ready answer: Retain durable event history, periodically recompute authoritative customer metrics for a bounded range, compare them with serving results and apply idempotent corrections.

20 What is your schema evolution strategy?

Interview-ready answer: Version the event contract, enforce producer compatibility through a registry, allow reviewed additive changes and quarantine incompatible payloads.

21 How would you choose Kafka partition keys?

Interview-ready answer: For per-customer ordering I would key by customer_id, while balancing skew and required throughput. I would measure hot customers before finalizing partition count.

22 How would you control state growth?

Interview-ready answer: Use event-time watermarks and bounded state, avoid unbounded group state, monitor state-store size and choose checkpoint storage with adequate throughput.

23 Why is payload a string in Spark?

Interview-ready answer: It keeps the compact schema flexible, but it sacrifices typed validation. Production should define versioned payload schemas by event type or preserve raw JSON plus normalized fields.

24 What would fail first at high volume?

Interview-ready answer: Small-file growth, checkpoint/storage throughput or partition skew are likely early issues. The offline pandas aggregation would also fail if used on large history.

25 What is the first production change?

Interview-ready answer: Persist invalid records and establish live deduplication/late-data rules, because correctness and recoverability must be defined before scaling.

26 Did you deploy this project?

Interview-ready answer: I describe it as a working portfolio implementation with passing unit CI, not as a production deployment. The repository proves specific code paths; a staging environment and integration evidence are still required.

Interview Questions (continued)

27 What exactly did you test?

Interview-ready answer: I tested Python event contracts, offline quality failures, customer aggregation and AI configuration. Those tests run offline and isolate deterministic behavior, so I do not claim they validate live external services.

28 Why does green CI not prove the platform works?

Interview-ready answer: CI executes syntax and unit tests, but it does not automatically prove networking, IAM, service configuration, runtime compatibility or end-to-end data correctness. I would add integration and staging smoke tests for that evidence.

29 What is the biggest gap?

Interview-ready answer: The most important gap is silent loss of invalid streaming rows and absence of live deduplication. I would address it before calling the system production-ready.

30 How do you handle secrets?

Interview-ready answer: The repository should contain examples only. I would use workload identities or IAM roles for cloud access and a managed secret backend for database credentials, with least privilege and rotation.

31 How do you handle logging?

Interview-ready answer: I would use structured logs with run or batch identifiers, input and output counts, durations, quality results and error categories. I would avoid raw customer data and make logs searchable in a central platform.

32 How do you monitor data freshness?

Interview-ready answer: I would record the newest processed event or business timestamp and compare it with the freshness SLA. Breaches should alert with the affected dataset and last successful run.

33 How do you make retries safe?

Interview-ready answer: Every side effect needs an idempotency key or deterministic batch identifier. I would isolate run outputs, use upserts or transactional publication, and ensure a retry cannot duplicate or overwrite another run.

34 How do you support backfills?

Interview-ready answer: I would parameterize a bounded time range, use run-specific outputs, limit concurrency, reconcile every partition, and record completion in a control table so a failed slice can be resumed safely.

35 How do you prevent schema drift?

Interview-ready answer: I would define an explicit versioned contract, validate it at ingestion, quarantine incompatible records, test producer-consumer compatibility and allow only reviewed evolution.

Interview Questions (continued)

36 How would you improve CI?

Interview-ready answer: I would keep fast unit checks, add linting and dependency scanning, then add targeted integration jobs using ephemeral infrastructure. Live staging smoke tests should run separately with tightly scoped credentials.

37 How would you test external services?

Interview-ready answer: I would combine client stubs for request contracts, local or containerized substitutes where accurate, and a small real staging test for service-specific behavior that emulators cannot prove.

38 How do you protect customer data?

Interview-ready answer: I would minimize collected fields, encrypt in transit and at rest, restrict access by role and prefix/table, mask sensitive fields, set retention policies and audit access.

39 How do you estimate capacity?

Interview-ready answer: I would measure arrival rate, record size, daily volume, peak concurrency, transformation cost and downstream throughput. Those numbers drive partitioning, compute sizing and retention choices.

40 What happens when data quality fails?

Interview-ready answer: Production should stop publication for critical failures, persist rejected records with reason codes, alert the owner and support corrected replay. Warning-level rules can continue with explicit metrics.

41 How do you prove lineage?

Interview-ready answer: I would attach run ID, source window, input location, code version and output location to a run manifest and integrate those records with a catalog or lineage platform.

42 What would you document for operations?

Interview-ready answer: I would document SLAs, owners, dependencies, restart and backfill procedures, common failure signatures, data contracts, access requirements and rollback steps.

43 What would fail first at scale?

Interview-ready answer: The first bottleneck is likely small files/checkpoint pressure in streaming and the offline pandas aggregation if applied to full history. I would validate that with load tests and redesign before increasing volume.

44 How do you control cost?

Interview-ready answer: I would right-size compute, prune partitions, compact small files, apply storage lifecycle rules, cap retries and timeouts, and publish per-run cost and volume metrics.

Interview Questions (continued)

45 What would you change in your first week?

Interview-ready answer: I would first add evidence around the highest-risk path: add quarantine, streaming deduplication/watermarking, a runnable entry point and Kafka/Spark restart tests. Then I would update the runbook and architecture based on measured results.

46 How do you explain a limitation without weakening your interview?

Interview-ready answer: I state the boundary directly, explain why the current choice was reasonable for a portfolio scope, cite what is actually tested, and give a concrete production solution.

47 Why should an interviewer value this project?

Interview-ready answer: It shows that I understand the complete data lifecycle and can review my own design critically. I can discuss implementation, tests, failure modes and realistic hardening rather than only drawing an architecture.

48 How would you perform a production readiness review?

Interview-ready answer: I would review correctness, recoverability, security, scalability, observability, deployment, cost and ownership. Each requirement needs evidence, not only code presence.

49 What is your rollback strategy?

Interview-ready answer: I would version code and data outputs, publish atomically, retain the previous valid version, and make consumers switch through a controlled pointer or view. Rollback steps must be tested.

50 Give the honest one-line status.

Interview-ready answer: This is a compact, technically useful portfolio implementation with verified unit behavior and clearly documented production gaps; it is not a fully deployed platform.

Rapid Revision Sheet

One-minute memory map

- > Flow: producers -> Kafka -> Spark -> valid date-partitioned Parquet -> dbt/pandas aggregation.
- > Checkpoint exists as configuration; restart behavior is not integration tested.
- > Invalid stream rows are filtered, not quarantined.
- > Live duplicate and late-event handling are not implemented.
- > dbt expects an external silver source; Terraform covers S3 only.
- > Testing truth: pure Python CI, not live Kafka/Spark.

Safe sentence when challenged

The repository demonstrates the design and unit-tests specific offline behavior. It does not prove every external integration or a production deployment. I can show exactly what is verified and what I would add to prove the remaining path.

Glossary

Term	Meaning
Kafka	Distributed retained event log.
Topic	Named Kafka event stream.
Partition	Ordered shard of a Kafka topic.
Structured Streaming	Spark API for incremental stream processing.
Checkpoint	Durable offsets and state for restart.
Event time	Time the business event occurred.
Ingest time	Time the pipeline observed the event.
Watermark	Boundary for late event/state cleanup.
Customer 360	Consolidated customer-level representation.
dbt	SQL transformation and testing framework.